```
$ procura --review
```

PROCURA

Scheduling personal tasks with operating-system policies, grounded AI diagnosis, and self-healing recovery

[Your Name] · Operating Systems · github.com/harshitchhabi/procura

$ cat problem.txt

To-do apps ignore forty years of scheduling research

Priority is a label

A task is High, Medium or Low because the user said so. Nothing combines deadline urgency, time already waited and cost into an actual order.

OS answer: scheduling policies

Fairness isn't measured

Whether attention is spread evenly across competing tasks is assumed, never reported.

OS answer: Jain's Fairness Index

Failures aren't handled

A blocked or starved task shows a red flag. Nothing explains why it failed, and nothing repairs it.

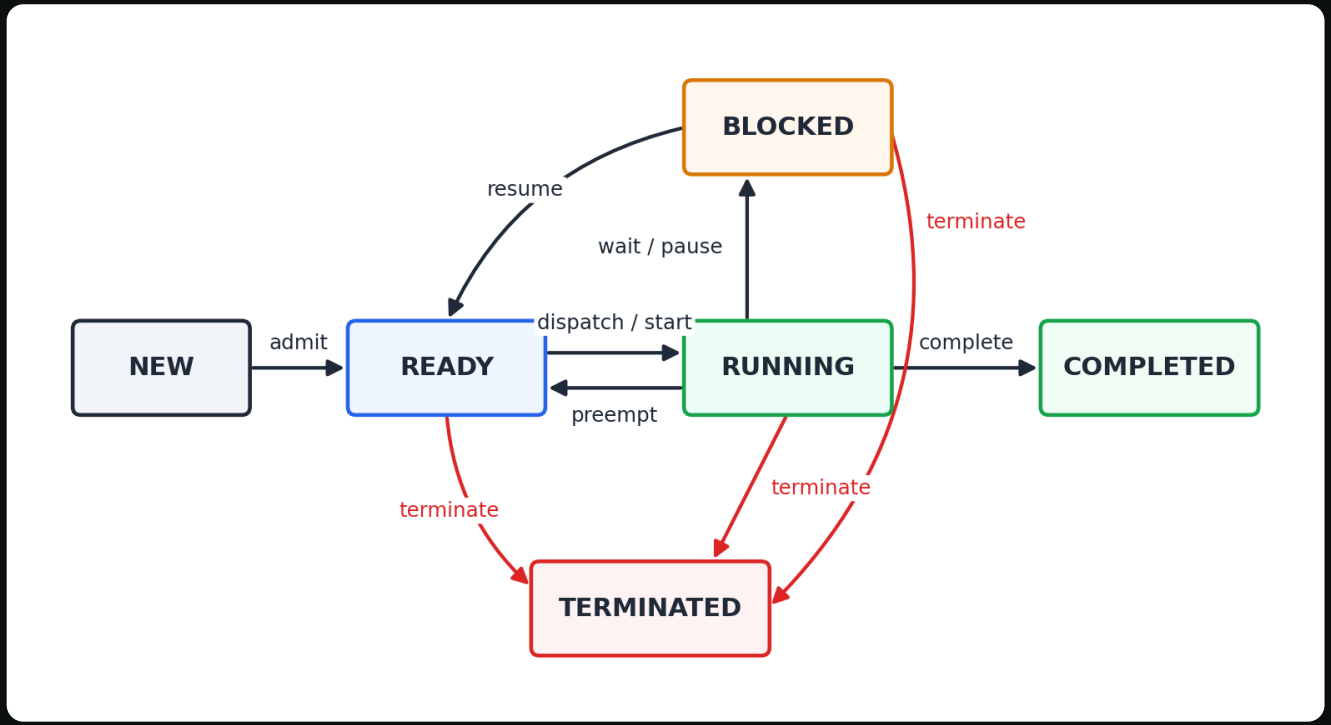
OS answer: self-healing loop

Procura's question: can these mechanisms be applied faithfully — same formulas, same state machines — to personal tasks?

$ procura --model

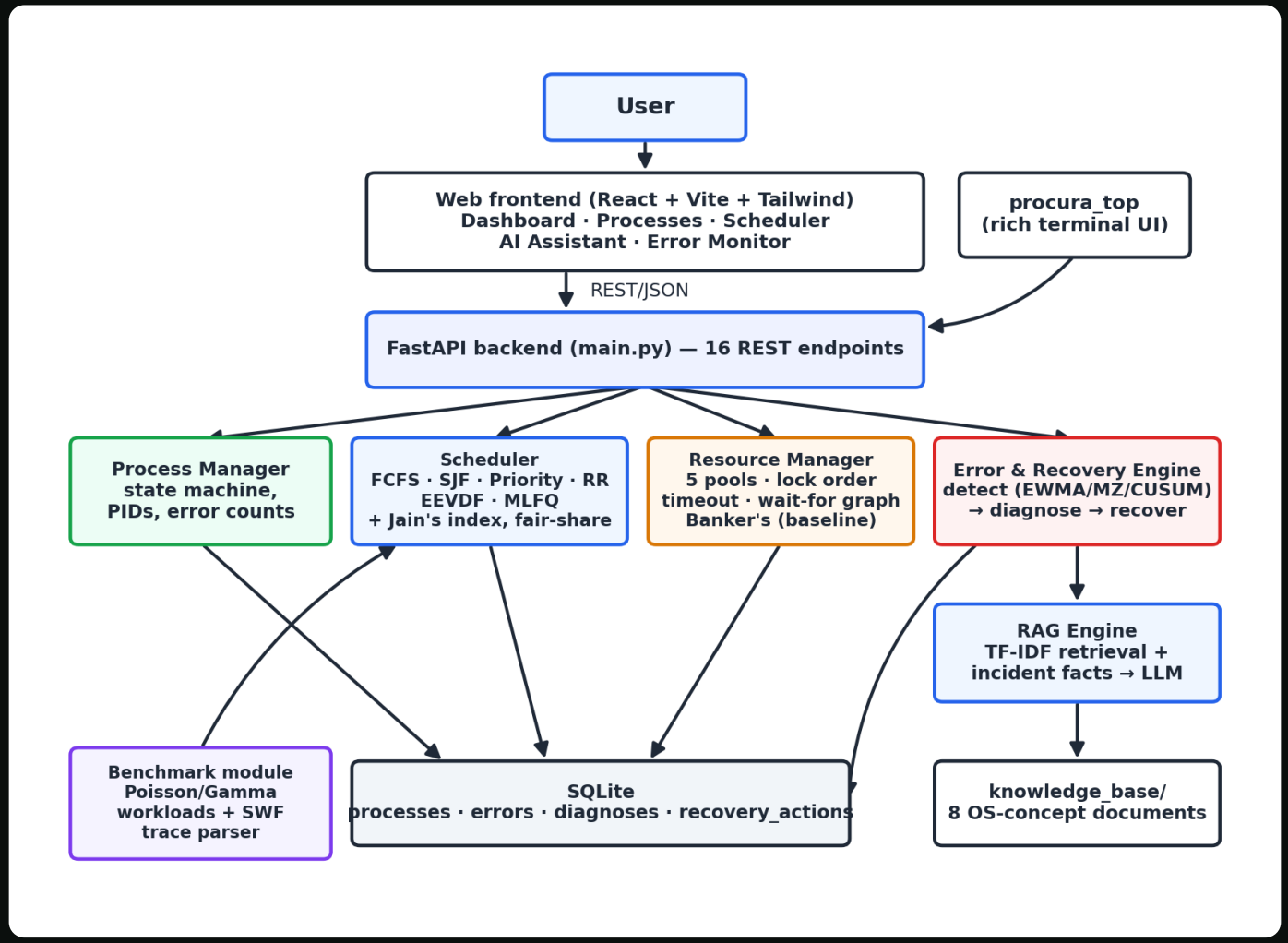
Every task is a process

Task	→	**Process with a PID**
Estimated effort	→	**Burst time (minutes)**
Importance + deadline	→	**Effective priority**
Memory / CPU needed	→	**Resource requests**
Stuck on someone else	→	**Wait-for edge**



$ procura --architecture

One backend, four subsystems, two interfaces



Local-first

No account; works fully offline

Explainable

AI answers grounded in the system's own logs

Safe automation

AI explains; fixed scripts act

Verified

83 automated tests, all passing

$ procura schedule --list

Six scheduling policies, from textbook to production

FCFS

textbook

Runs tasks in arrival order. Simple; long tasks delay everyone behind them.

SJF

textbook

Shortest arrived task first. Lowest average wait; long tasks can starve.

Priority

Slurm-inspired

Five-term effective priority with aging and Slurm-style fair-share decay.

Round Robin

textbook

Every task gets a 4-minute turn in rotation. Bounded wait between turns.

EEVDF

Linux since 6.6

Weighted fair sharing: eligible tasks by earliest virtual deadline.

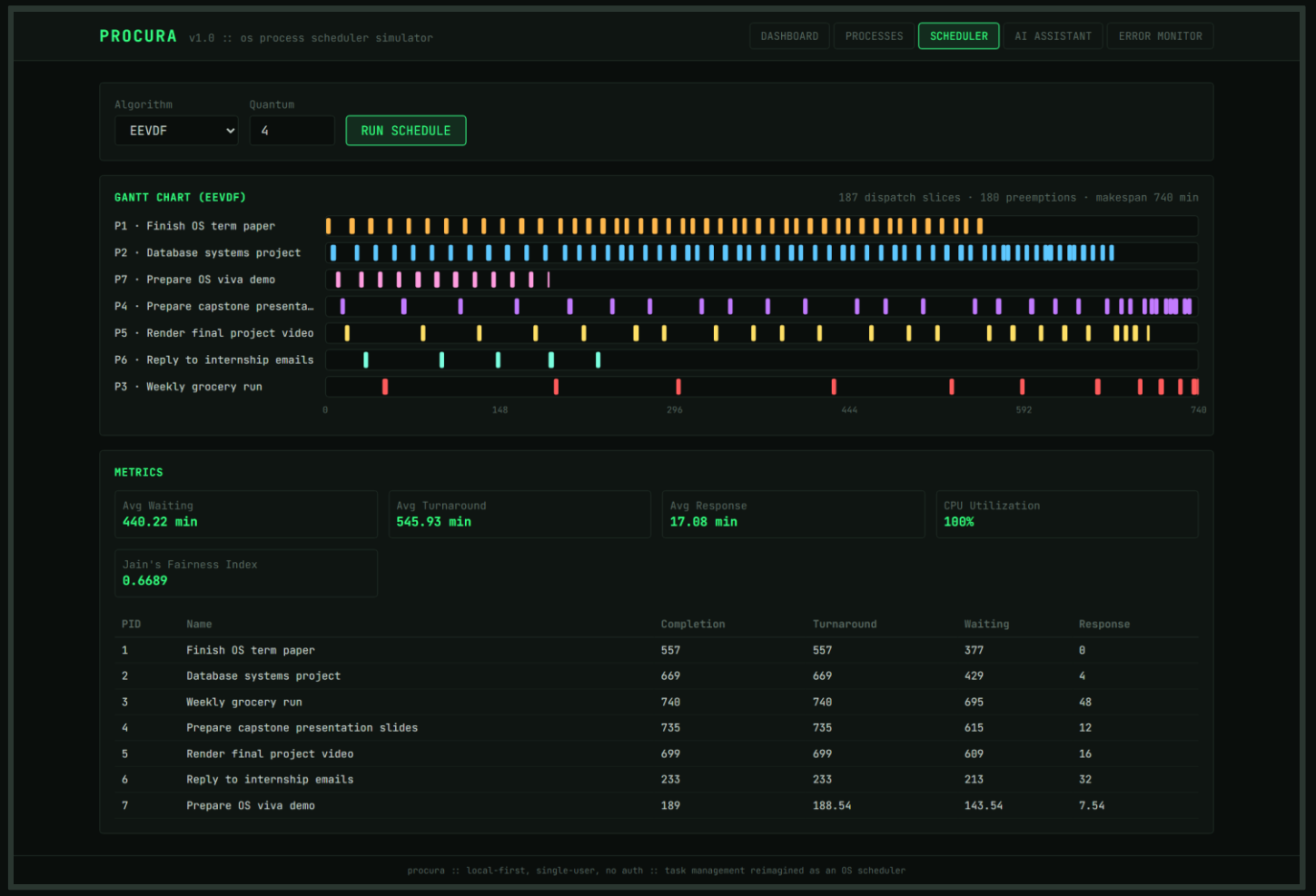
MLFQ

OSTEP

Three queues, demotion on allotment use, periodic boost against starvation.

$ procura schedule --algo eevdf

EEVDF turns priority into weighted sharing



187

dispatch slices on a 740-minute timeline

0 min

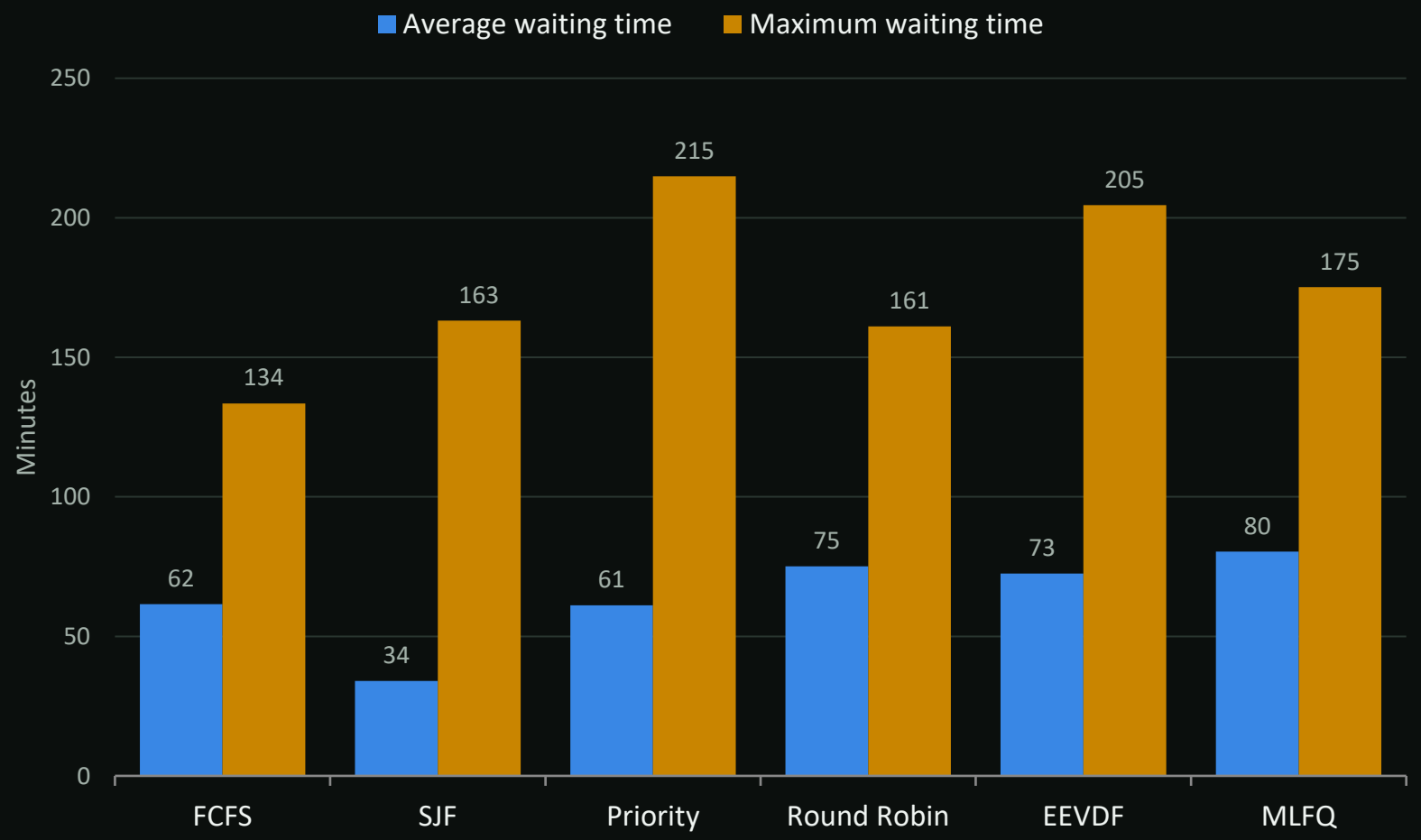
until the High-priority term paper first runs

~48 min

until the Low-priority errand runs — but it always does

```
$ python -m benchmark.run_benchmark --seed 42
```

SJF minimises the average wait — not the worst case



34 min

SJF average wait — lowest of all six

163 vs 134

SJF vs FCFS worst-case wait (min)

228.67

identical makespan: all six are work-conserving

$ procura schedule --compare --live

Preemption buys responsiveness with waiting time

431 min

FCFS — average time until a task first runs

173 min

SJF — non-preemptive, shortest first

12 min

Round Robin and MLFQ — every task starts almost at once

Average response time, seven tasks, 740-minute makespan for every policy

- All six policies reach 100% CPU utilisation — the difference is who waits, not how much work gets done
- Jain's Fairness Index is identical (0.6689) across all six: every task eventually gets exactly its burst time, so it measures how much, not when

$ procura detect

Detect → diagnose → recover, like AIOps platforms

01 DETECT

EWMA · modified Z-score · CUSUM (NIST)

Memory exhaustion

Wait-for-graph cycles

02 DIAGNOSE

TF-IDF retrieval over 8 OS documents

+ the incident's own logged facts

→ LLM, or offline answer

03 RECOVER

Deterministic, audited scripts

retry · suspend_and_retry

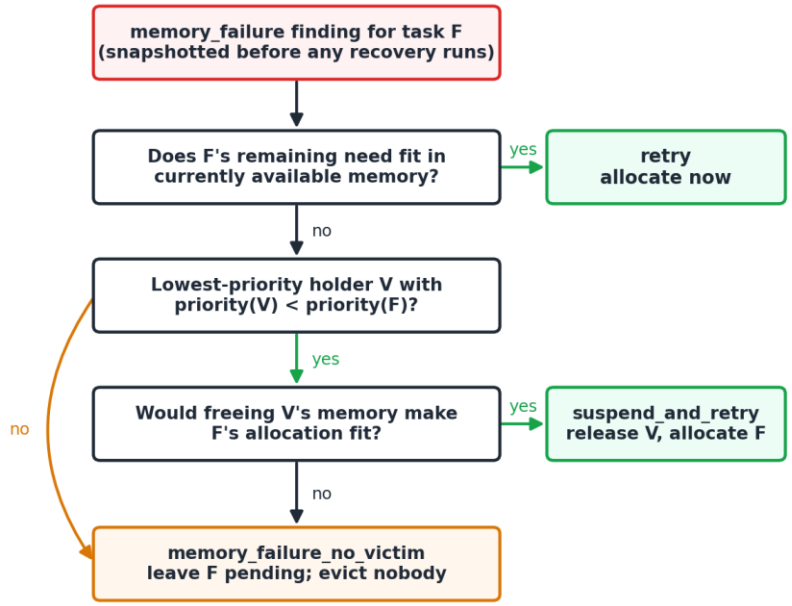
terminate cycle victim

The AI explains. A fixed script acts.

Every error, diagnosis and action is written to SQLite and shown in the Error Monitor — a full audit trail.

$ procura detect --memory

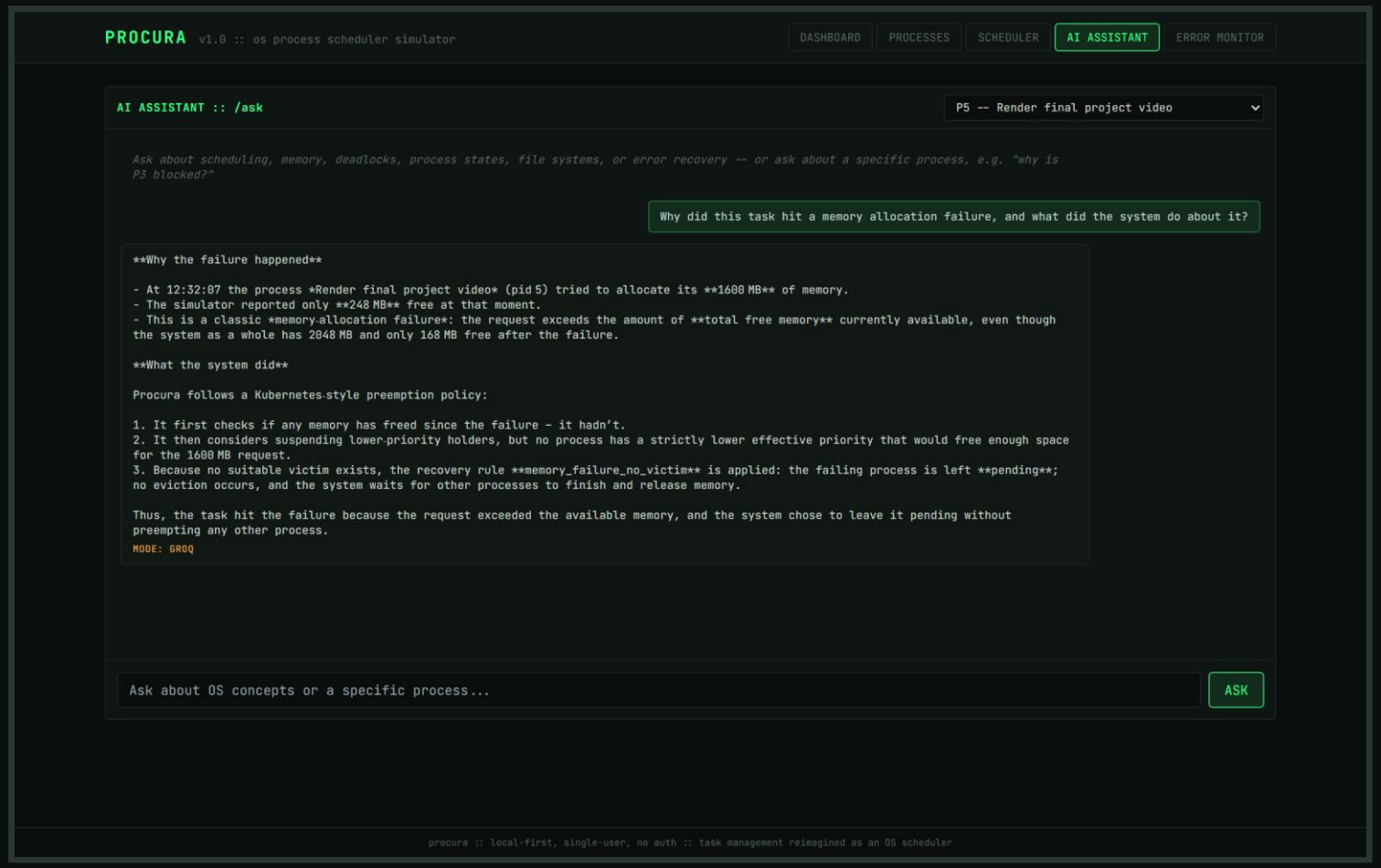
Recovery follows Kubernetes-style preemption rules



P2	768 MB	`suspend_and_retry` — evicts lower-priority P3; now fits
P4	256 MB	`retry` — memory freed by P2's recovery
P7	256 MB	`retry` — memory still available
P5	1600 MB	`no_victim` — nothing lower-priority frees enough

$ procura ask --pid 5

Grounding fixed the AI, not a bigger model



BEFORE

"Caused by memory fragmentation" — invented; never in the log

AFTER

1600 MB requested, 248 MB free; no lower-priority holder could free enough, so nothing was evicted

Same model. The difference was what it was given.

```
$ python -m pytest tests/ -v
```

Running the system found what unit tests missed

83

automated tests, all passing

12

defects found by running the system

1

published result inverted, then corrected

SJF idled the CPU

The test asserted the bug (avg wait 5.75 vs textbook 5.25)

Recovery ping-pong

Two tasks evicted each other; now strictly-lower priority only

AI invented causes

No incident data in the prompt; now injected as ground truth

Tests called a live LLM

load_dotenv re-added deleted keys; now blanked

$ procura --limits

Limitations and what comes next

Limitations

- A simulation on one abstract CPU, not a real kernel
- Small workloads: 30 synthetic jobs, a 15-job trace, 7 live tasks
- AI evaluated qualitatively; it can still overstate
- Single local user, no authentication by design

Future work

- Fairness over waiting time and allocation rate, not only totals
- Replay full Parallel Workloads Archive traces
- A labelled question set to score the assistant
- Multi-user fair-share with authentication

Each item is discussed with evidence in the documentation paper, Sections 11–12.

```
$ procura --summary
```

OS theory works on to-do lists — and makes its trade-offs visible

Scheduling

SJF minimises average wait, preemption buys responsiveness, EEVDF shares by weight without starvation

Self-healing

Principled preemption rules turn a cascading recovery engine into one whose every action is justified

Verification

Twelve defects survived passing unit tests — found only by checking the running system

Questions?

github.com/harshitchhabi/procura · docs/: paper, demo walkthrough, case study